

PTA-Viz

Siddharth Mahesh Patil
21d070071

April 2025

Contents

1	Introduction	3
2	Dependencies	3
3	Completed so far	3
4	Todos	3
5	User Manual	4
6	Scanner+Parser Code	4
6.1	stmt_helper.py	5
6.1.1	elem_types and stmt_types	5
6.1.2	stmt	5
6.1.3	elem	5
6.2	scanner.py	6
6.3	parser.py	6
6.3.1	parsetab.py, parser.out	8
7	Code for analyses	9
7.1	pta_helper.py	9
7.1.1	get_fspta_stmts, get_pta_stmts	9
7.1.2	set_ptr_dict	9
7.1.3	get_def(s), get_pointee(s)	9
7.1.4	unify	9
7.1.5	get_must_pt, get_strong_update	9
7.1.6	set_pin, set_lb_pin	10
7.1.7	set_pout, set_lb_pout	10
7.1.8	get_ref, set_lhsref, set_lhsrhsref	10
7.1.9	set_lin, set_lout	10
7.2	helper.py	10
7.2.1	update_count	10

7.2.2	invert_dict	10
7.2.3	save_dict_to_json	10
7.2.4	save_in_out_dicts_to_json, save_in_out_inverted_dicts_to_json	10
7.2.5	nested_len_pt, nested_len_l	10
7.2.6	get_stmt_graph	11
7.2.7	contains_pointer	11
7.2.8	get_next_counter	11
7.2.9	save_points_to_graph	11
7.2.10	get_points_to_graph_from_file	11
7.3	fi_pta.py	11
7.3.1	perform_anderssens_analysis	11
7.3.2	perform_steensgaards_analysis	11
7.4	fs_pta.py	12
7.5	lfcpa.py	12
7.5.1	preform_lbpta	12
7.5.2	preform_ptbla	12
7.5.3	perform_lfcpa	12
7.6	vasco_pta.py	12
7.7	main.py	12
8	Code for the GUI	12
8.1	guiHelper.py	13
8.2	gui_editor.py	13
8.2.1	QCodeEditor	13
8.2.2	QCodeEditorWindow	13
8.3	gui_imageViewer.py	13
8.3.1	QScrollableImage	14
8.3.2	QCFGWindow	14
8.3.3	QPTAWindow	14
8.3.4	QLFCPAWidget	15
8.4	gui.py	15

1 Introduction

This is a report for my RnD project [PTA-Viz](#).

The source code could be found here - <https://github.com/Siddharth-011/RnD>.

In this report, I have explained all parts of the code I have written so far, what is completed and what has to be done.

The report is split into sections based on the use of the code, i.e. scanner+parser, analysis and GUI, and in each section, I describe the uses of each and every function of every file in these sections.

2 Dependencies

To run the code, the user will need to install the following dependencies:

- Python (>3.10)
 - [PLY](#) (pip install ply)
 - [graphviz](#) (pip install graphviz)
 - [PyQt6](#) (pip install PyQt6)
- System apps
 - [Graphviz](#) (Graphviz app needed along with the graphviz python library)

3 Completed so far

I have completed the following:

- Code for scanner+parser, intra-procedural FI-PTA (Andersen's and Steensgaard's), FS-PTA and LF CPA.
- Code for storing the analysis results of all the above analyses except Steensgaard's Analysis.
- Visualization of PT information using Graphviz.
- GUI code editor, CFG viewer and LF CPA results viewer using PyQt.

4 Todos

I have listed the work to be done in decreasing order of priority:

1. Complete the code for VASCO PTA and inter-procedural LF CPA.
2. Write code to store the results for Steensgaard's FI-PTA, VASCO PTA and inter-procedural LF CPA.

3. Make Qt widgets to display the results of all the analyses using the QLFC-PAWidget as reference.
4. Store the exact equations used in each step of the analysis and display the exact rules used in the GUI.
5. Implement a way to "run" the analysis, i.e. a play button which will automatically start running the analysis step-by-step in the GUI.
6. Allow importing and exporting the analyses results.

5 User Manual

To run all the analyses and store their results as json files, without using the GUI, first set the "file_name" variable in main.py to the program file to be analyzed and then run:

```
$ python3 main.py
```

To run the entire program through a GUI, run:

```
$ python3 gui.py
```

Then type the code in the GUI code editor, or, alternatively, directly load a program file using the "Open" button.

To perform the analysis, press the "Analyze" button, and the CFG and the results should show up in the adjacent widgets. On performing the analysis, the results are stored as json files.

The CFG displayed is clickable and the user can click on any statement/block to see the analysis results at that program point (both in-information and out-information is displayed simultaneously).

In the results widget, there are spinboxes labeled "Iteration" and "Round" which could be used to see the analysis results at specific iteration and round.

For LFCPA, there is another button at the top right of the results widget which could be used to toggle the analysis type, i.e. if the button displays "LA", then the results being shown are for the liveness round and if it shows "PTA", then the results being displayed are for the PTA round. Pressing this button toggles the analysis type.

Apart from this, every text box has a "Font" spinbox to change the text font, every image window has a "Magnification" spinbox to adjust the magnification of the image and every widget is re-sizable, i.e. the user can drag the borders of the widgets to expand, shrink or collapse them.

6 Scanner+Parser Code

The following files are used for scanning and parsing:

6.1 stmt_helper.py

This file contains the classes used to store the statements and statement's elements.

The following are the main classes and their functions:

6.1.1 elem_types and stmt_types

These classes are used to store the different types of statements and statement elements.

6.1.2 stmt

Base class for all statement classes.

Functions:

- `get_display_stmt`: returns the statement as a formatted string for displaying.
- `is_stmt_type`: checks the statement type.

Inherited by:

- `plain_text`: used to store the "START" and "END" statements (for CFG)
- `assignment`: used to store assignment statements
- `input`: used to store "read" statements
- `use`: used to store "use" statement
- `goto`: used to store the "goto" statement
This is inherited by `cond`: used to store the "if" statements
- `call`: used to store the function calls

6.1.3 elem

Base class for all statement element classes.

Functions:

- `get_display_value`: returns the element as a formatted string for displaying.
- `is_elem_type`: checks the statement element type.
- `elem_type`: getter function to get the element type (useful in switch cases).

Inherited by:

- `number`: Store numbers (scalars)
- `boolExp`: Used to store boolean expressions

- variable: Used to store variables and is the base class for the following:
 - address: Used to store "&<varName>"
 - malloc: Used to store "malloc()"
 - pointer: Used to store "*<varName>"
 - field: Used to store "<varName>.<field>".
This is also the base class for fieldPointer: Used to store "<varName>-><field>".

6.2 scanner.py

Scans the used input and send the tokens to parser.py.

Variables:

- reserved: Stores the reserved keywords dictionary (keyword:token).
- literals: The symbols which are passed as it is to the parser.
- tokens: List of tokens sent to the parser.
- t_LTE: Regex to match LTE
- t_GTE: Regex to match GTE
- lno: Used to track the line number

Functions:

- t_STARS: Used to match consecutive '*'s. Returns the string as it is.
- t_VARNAME: Used to match valid varNames. Returns a "variable" object.
- t_NUMBER: Used to match scalar inputs. Returns a "number" object.
- t_NEWLINE: Used to match newlines and comments. Returns the number of lines covered and also updates lno.
- t_SPACES: Used to match spaces.
- t_error: Raises an exception if the user input cannot be tokenized.

6.3 parser.py

Parses the list of tokens returned by the scanner.

Variables:

- precedence: Defines the precedence of tokens.
- statements: Used to store the list of statements in every function.

- `fno`: Store the line number of the start of the function.
- `lno`: Stores the line number of the current line.
- `lnomap`: Stores the `lno` to `alno` map (as a list).
- `alno`: Actual line number of the statements (after removing extra newlines etc).
- `defstructlist`: List of defined structures.
- `usedstructlist`: List of used structures (for error checking).
- `deffunclist`: List of defined functions.
- `usedfunclist`: List of used functions (for error checking).
- `funcdict`: Stores the information about every function (`func_id`:`[func_params, func_var_dict, func_stmt_list]`).
- `varlist`: List of defined variables (used for error checking).
- `structdict`: Dictionary to store structure's information (`struct_name`:`[dict(field:field_type), list(fields)]`).
- `vardict`: Dictionary to store variable types (`var_name`:`var_type`).
- `start`: Parser start point.

Regex Matching Functions:

- `p_space`: matches a space or empty string.
- `p_bool_op`: Matches boolean operators with space and returns the boolean operator.
- `p_bool_exp`: Matches boolean expressions and returns a `boolExp` object (+ type).
- `p_lhs`: Matches valid LHS, except "VARNAME" and returns the appropriate `elem` object (+ type).
- `p_rhs`: Matches valid RHS and returns the appropriate `elem` object (+ type).
- `p_var_dec`: Matches variable declarations and updates `varlist` and `vardict`.
- `p_stmt`: Matches statements, returns the appropriate `stmt` object and updates statements, `alno` (and `vardict` if the statement is `malloc`).
- `p_tac`: Matches the main function and updates the `fno`.
- `p_nl`: Matches newline and updates the `lno`.

- `p_stmt_nl`: Matches newlines in statements and updates `lno` and `lnomap`.
- `p_funcbody`: Matches function body and updates `lnomap` if necessary.
- `p_func_args`, `p_arg_list`: Used to match the function argument list in function calls (with type checking).
- `p_func_params`, `p_param_list`: Used to match the function parameter list in function definitions.
- `p_func`: Matches the list of functions, except the main method.
- `p_space_nl`: Matches space or newline
- `p_list`: Matches a list of variables (without type checking)
- `p_dec_list`: Matches a list of variable declarations in a structure.
- `p_structs`: Matches a list of structure definitions.
- `p_prog`: The rule for the entire program.

Functions used in error checking:

- `p_error`: Raises exceptions in-case of parsing errors.
- `checkvar`: Returns the type of the variable, if it is defined, otherwise throws an error.
- `checktype`: Checks the type of the error using `checkvar` and returns an error if the variable does not exist or the types do not match.
- `checkfield`: Checks the existence of a given fields in a variable and returns an error if the variable or the field does not exist.
- `checkptrfield`: Same as `checkfield`, but checks pointer fields (`var->fld`)

Helper functions:

- `prepare_next_function`: Prepares the global variable for the beginning of a new function.
- `clean_statements`: Cleans up the statement list and modifies the `goto/condition` line number to the `also`.
- `reset`: Resets the current state to facilitate parsing of new input programs.
- `parse_file`: Parses a file.
- `parse_text`: Parses text input.

6.3.1 `parsetab.py`, `parser.out`

Files created when we call `parser.py` for the first time.

7 Code for analyses

The following are the helper files used for analysis/visualization of the the results:

7.1 pta_helper.py

The main helper code for PTA.

7.1.1 get_fspta_stmts, get_pta_stmts

To get the updates statement list for FS-PTA and FI-PTA. It removes the "goto" statements and only keeps statements which have any kind of pointer manipulation. The "lb" argument of "get_fspta_stmts" is to identify if there will be liveness-based analysis, in which case we will also need to include scalar assignments in our statement list if the lhs or rhs is de-referencing a pointer. It returns the new_stmt_list (and the successors and the predecessors for FS-PTA).

7.1.2 set_ptr_dict

Helper function to set the initial Pointer dictionary. If "enable_unk" is true, then it sets the initial PT information to "?" for every (varName, field) pair, otherwise, it keeps the initial PT information empty, i.e. ptr_dict[var][field]={} for all (varName, field) pairs.

The type of the ptr_dict is PointerDict = dict[str, dict[str, set[str]]] and the PT information is stored as ptr_dict[var][field] = set of pointees.

7.1.3 get_def(s), get_pointee(s)

get_def, get_pointee are used by Steensgaard's FI-PTA, and get_defs, get_pointees are used by all other analyses.

get_def(s) uses equation 4.7 from Sir's book to return var(s) and field.

get_pointee uses equation 4.8 from Sir's book to return pointee(s).

7.1.4 unify

Function to recursively unify sets of variables.

sets_to_unify is a tuple of the representative element of the two sets to unify.

It unifies sets_to_unify by modifying var_to_set_dict and set_to_var_dict and checks for cascading effects and stores the new_sets_to_unify as a dictionary and recursively calls unify on these.

7.1.5 get_must_pt, get_strong_update

get_must_pt computes and returns MustPT using equation 4.15 from Sir's book.

get_strong_update uses get_must_pt to return vars and field according to equation 4.12 in Sir's book.

7.1.6 set_pin, set_lb_pin

Used to set the Pin using equations 4.16 and 6.5 respectively, from Sir's book.

7.1.7 set_pout, set_lb_pout

set_pout takes ptr_dict_in as input and overwrites it with ptr_dict_out.

Used to set the Pout using equations 4.17 and 6.6 respectively, from Sir's book. Internally computes kill and gen using equations 4.13 and 4.9 respectively, from Sir's book.

7.1.8 get_ref, set_lhsref, set_lhsrhsref

Use equations 6.1 and the next two unnumbered equations, respectively, from Sir's book, to compute Ref.

7.1.9 set_lin, set_lout

Uses equations 6.3 and 6.2 respectively, from Sir's book, to compute Lin and Lout.

7.2 helper.py

This file contains more general helper functions.

7.2.1 update_count

Function used to update the color count used to color the nodes of the PT graph.

A function is needed because the color scheme used has yellow in it which is not clearly visible in a white background, therefore this function skips the yellow color.

7.2.2 invert_dict

Inverts the input dictionary. Used for storing the liveness dicts.

7.2.3 save_dict_to_json

Saves the given dict as a json file.

7.2.4 save_in_out_dicts_to_json, save_in_out_inverted_dicts_to_json

Used to store the PT information and liveness dicts as json files respectively. Input is a list of tuples of in and out dicts, i.e. set[(in_dict, out_dict)].

7.2.5 nested_len_pt, nested_len_l

Computes the nested length of ptr_dicts and liveness_dicts respectively.

7.2.6 `get_stmt_graph`

Constructs the CFG (or stmt list graph for FI-PTA) using graphviz, saves it as a svg file and returns the bounding boxes of the nodes of the graph.

7.2.7 `contains_pointer`

Returns true if the given type contains any pointer, i.e. if the type is a pointer or if it is a structure, then does that structure contain any field which is a pointer.

7.2.8 `get_next_counter`

Helper function used while computing the predecessors and successors when we trim the statement list.

7.2.9 `save_points_to_graph`

Creates the points to graph from ptr_dict using graphviz. The graph is either saved as an image, or returned as a svg file.

7.2.10 `get_points_to_graph_from_file`

Reads the ptr_disct from the given file and, construct and returns the points to graph in svg format.

The following are the files used to perform the PTA:

7.3 `fi_pta.py`

This file contains the code for Andersen's and Steensgaard's FI-PTA.

7.3.1 `perform_andersens_analysis`

This function performs Andersen's FI-PTA using the algorithm mentioned in figure 4.1 in Sir's book.

The function also saves the analysis results for each iteration.

7.3.2 `perform_steensgaards_analysis`

This function performs Steensgaard's FI-PTA. The algorithm for this is very similar to that of Andersen's FI-PTA, with the addition of unification of variables.

The function only displays the final points to graph and the code to save the PT graph at each iteration has to be written.

7.4 fs_pta.py

This file contains a single function, "perform_fspta", which is used to perform FS-PTA using the Round-Robin algorithm described in figure 4.8 in Sir's book. The code saves the PT information for every node for every iteration.

7.5 lfcpa.py

This file contains the functions used to perform intra-procedural LFCPA based on the algorithm in figure 6.5 of Sir's book.

7.5.1 preform_lbpta

Performs PTA based on the liveness information at each node. The PT results in every round are stored for every node.

7.5.2 preform_ptbla

Performs liveness analysis based on the PT information at each node. The Liveness results in every round are stored for every node.

7.5.3 perform_lfcpa

Performs LFCPA by iteratively calling preform_lbpta and preform_ptbla until there is no change.

This function also handles empty (no statement) loops which could be created by chained goto statements using a "loop" statement. We did not have to account for this in FS-PTA, but have to here because of the liveness analysis which starts from the function's end statement. The last statement of the statement list is the "END" statement, unless there is a loop in the program, in which case the last statement is the loop statement.

7.6 vasco_pta.py

Work in Progress

7.7 main.py

Main file for performing analysis. It has a single function, "perform_analysis", which creates the result directories and calls all the analyses functions.

8 Code for the GUI

The following are the files used to create the GUI:

8.1 guiHelper.py

This file is used to "overwrite" some default QTWidgets to add some repeated functionality.

- QPushButton: A standard QPushButton with a one line initialization and fixed size.
- QSpinBox: A standard QSpinBox with a one line initialization, fixed size and a reset function with changes the range and value of the spinbox simultaneously.
- QSplitter: A standard QSplitter with a wider handle.
- QLabel: A standard QLabel with a fixed width.
- QTextWidget: A non-editable text window which can set text from a dictionary (used to display liveness at program points) with the ability to change the text font. The setText function allows us to change the displayed text and the function updateFontSize can be used to change the text font.

8.2 gui_editor.py

This has the code for the GUI code editor.

Note that here I have not described all the functions because they are working as expected and do not need any change.

8.2.1 QCodeEditor

Inherits the QPlainTextEdit Widget and adds a number bar, current line highlighting and the ability to update the text font.

The number bar is created using the NumberBar class which inherits QWidget. The implementation of the code editor is heavily inspired by [this](#).

8.2.2 QCodeEditorWindow

The main widget of the code editor.

It uses the QCodeEditor to create the editor widow and also has additional functionality to open, save, saveas and analyze the code along with the ability to change the font size and display error messages.

8.3 gui_imageViewer.py

This file contains the code for visualization of the CFG and the results (only LFCPA for now).

8.3.1 QScrollableImage

This widget can display zoom-able images and also allows one to highlight certain sections of the image. This is used to display all the graphs.

Internally uses QImage, which inherits QSvgWidget, to display the svg images stored during analysis.

- QImage: Widget to display svg images. Has the following functions:
 - paintEvent: Function called whenever the image needs to be updates. Displays the original image and draws the highlight box if there is some box to be highlighted.
 - getPos: A function called on every mouse click. This function get the location of the click and if it on any graph node, then it sets the bounding box of that node as a highlight box and repaints the image.
 - setMagnification: Used to zoom the image.
 - setImage: Updates the displayed image.
 - setData: Updates the displayed image along with the bounding boxes of the nodes.
 - setHi: Sets the node to be highlighted.
- setScale: Sets the magnification of the QImage object.
- resetImage, resetData, setHi: Call the setImage, setData, setHi functions of the QImage object.

8.3.2 QCFGWindow

A Widget used to display the CFG.

- imageView: QScrollableImage object which is used to display the CFG.
- zoomSpinBox: QSpinBox object to update image magnification.
- resetData: Used to reset the displayed image and the bounding boxes of the nodes of the CFG.
- setHighlight: Set the node to be highlighted (Useful when displaying the statement list for FLPTA).

8.3.3 QPTAWindow

Widget to display the PT graph.

- imageView: QScrollableImage object which is used to display the PT graph (no bounding boxes provided).
- zoomSpinBox: QSpinBox object to update image magnification.
- resetImage: Resets the displayed image using the file path to the new PT information's json file.

8.3.4 QLFCPAWidget

Widget used to display the results for intra-procedural LFCPA.

The important objects are:

- PTAinNext, PTAoutNext, PTAinOld, PTAoutOld: QPTAWindow objects used to display the PT information at the entry and exit of the selected node for the current round and the previous round.
- LinNext, LoutNext, LinOld, LoutOld: QTextWidget objects used to display the liveness information at the entry and exit of the selected node for the current round and the previous round.
- iterSpinBox, roundSpinBox: QSpinBox objects to update the iteration and round of the displayed results.
- switchAnalysisButton: QPushButton object to change the displayed analysis type (liveness analysis, LA, or PTA).

The functions implemented are:

- getPath: Returns the path of the results of the current iteration and round.
- changeIter, changeRound: Used to change the current iteration and round respectively, and also updates the displayed results to match the current iteration and round.
- setData: Used to reset the widget to display the analysis results for a new program. This sets the number of iterations and rounds and displays the liveness results for the "START" statement at iteration 1, round 1.
- switchAnalysisType: Function which switches the analysis type from LA to PTA and vice versa and also updates the displayed widgets to match the analysis type.
- setCurrStmt: Sets the current statement and updates the displayed results to match the current statement.

8.4 gui.py

This is the main GUI file which displays the entire GUI.

This contains a single widget, "QApp" with the following objects and functions:

- editor: QCodeEditorWindow object whose analyze button is linked to the analyzeFunc function.
- results: QLFCPAWidget object to display the LFCPA results.
- cfgWindow: QCFGWindow object which is linked to the results.setCurrStmt method and calls this function whenever the user clicks on a different statement.

- `changeAnalysisType`: Function to change the analysis type, i.e. Andersen's, Steensgaard's, LFCPA, etc.
- `analyzeFunc`: Function to perform analysis on the given program.